

软件过程与管理课程作业

作业编号: Assgt_01

作业性质: 课后作业

提交人: 李欢 - 2023141461035

一、标准文件阅读心得摘要

在完成本次作业前, 已大致阅读并学习了课程群中发布的 ISO/IEC 12207、ISO/IEC 15504 及 ISO/IEC 33001 三份重要标准文档。

- **ISO/IEC 12207 (软件生命周期过程):** 理解了软件从概念提出到退役的整个生命周期中包含的协议过程、组织项目使能过程、技术管理过程和技术过程。
- **ISO/IEC 15504 & 33001 (过程评估模型):** 了解了如何对软件过程的能力进行评估和改进, 认识到过程定义并非一成不变, 而是需要根据组织和项目的成熟度进行动态调整。

二、个人项目软件过程定义

项目背景: “校园二手交易微信小程序”开发项目

团队规模: 5人 (1名产品兼项目经理, 1名UI设计, 2名全栈开发, 1名测试)

生命周期模型: 敏捷迭代模型 (Scrum)

结合项目实际情况, 我将该项目的软件过程定义如下 (映射至 ISO/IEC 12207 的核心过程组):

1. 技术管理过程 (Technical Management Processes)

- **项目策划过程:** 在每个迭代 (Sprint) 开始前进行 Sprint Planning 会议, 从产品待办列表 (Product Backlog) 中选取高优先级任务, 制定当前迭代的冲刺目标和开发计划。
- **项目评估与控制过程:** 通过每日站会 (Daily Scrum) 同步进度和障碍; 通过看板 (Kanban) 和燃尽图实时监控任务完成情况。
- **配置管理过程:** 使用 Git/GitHub 进行代码版本控制, 规定主分支 (Main)、开发分支 (Develop) 和特性分支 (Feature) 的合并规范。

2. 技术过程 (Technical Processes)

- **业务或任务分析过程:** 收集校园内学生二手交易的痛点, 定义小程序的核心业务场景 (如: 发布闲置、商品搜索、留言沟通)。
- **利益相关方需求定义过程:** 将业务场景转化为用户故事 (User Stories), 记录在产品待办列表中。
- **架构设计过程:** 确定前后端分离架构, 前端使用微信小程序原生框架, 后端采用 Spring Boot, 数据库采用 MySQL。
- **实现过程 (编码):** 开发人员按照分配的任务进行代码编写, 并编写基础的单元测试。

- 集成过程:后端提供 API 接口, 前端进行联调对接;采用持续集成(CI)工具进行代码的自动化构建。
- 验证与确认过程(测试):测试人员依据用户故事编写测试用例, 进行功能测试、兼容性测试;在每个迭代末尾进行 Sprint Review 会议, 由目标用户或指导老师进行系统确认(UAT)。
- 移交与运行过程:将小程序打包提交至微信公众平台审核, 发布上线, 并监控运行日志。

三、软件过程定义的来源与剪裁想法

1. 软件过程定义的来源

本项目的软件过程定义主要来源于 ISO/IEC 12207 国际标准中的“技术过程”与“技术管理过程”, 并深度融合了 Scrum 敏捷开发框架。

在现代软件工程实践中, 完全重型的标准过程往往显得臃肿, 因此我以 ISO/IEC 12207 提供的大框架为底层逻辑, 以敏捷思想作为执行指导, 确保过程既符合软件工程规范, 又能快速响应需求变化。

2. 过程剪裁(Tailoring)的想法与依据

按照 ISO/IEC 12207 的要求, 一个完整的软件生命周期包含数十个子过程。但由于本项目具有团队规模小、项目周期短、预算为零、需求变化快等特点, 我对标准过程进行了如下剪裁:

- 裁剪掉“协议过程(Agreement Processes)”
 - 剪裁依据:本项目为校园实践团队自研项目, 不存在甲乙方的招投标、正式合同签订及供应商采购管理环节。因此, 将“获取过程”和“供应过程”完全裁剪, 简化为内部的需求口头确认和简单的文档记录。
- 大幅精简“组织项目使能过程(Organizational Project-Enabling Processes)”
 - 剪裁依据:由于团队是临时组建的扁平化学生团队, 不具备复杂的组织级人力资源管理、基础设施管理等能力。我们将这一过程压缩至最低限度, 仅保留“开发环境搭建”和“组内技能分享”。
- 合并“系统分析”与“软件需求分析”过程
 - 剪裁依据:作为一款纯软件的小程序, 不存在复杂的软硬件系统级交互。因此, 省略了系统层面的需求与架构分析, 直接将业务需求转化为软件级的产品待办列表(Product Backlog), 减少了繁重的文档编写工作。
- 弱化“信息管理过程(文档化)”, 强调“工作软件”
 - 剪裁依据:遵循敏捷宣言“工作的软件高于详尽的文档”。我们裁剪了复杂的规格说明书编写, 仅保留核心的 API 接口文档(使用 Swagger 自动生成)、数据库字典以及 Git Readme, 以减轻开发者的文档负担, 将主要精力集中在功能交付上。

总结:通过对标准过程的合理剪裁, 我们在保证项目核心环节(需求、设计、编码、测试、配置管理)质量受控的前提下, 极大提升了小团队的开发敏捷度。这种“量体裁衣”的做法, 正是软件过程管理中灵活性与规范性相结合的体现。